Clipping 2

A. JEEWA

Parametric form of lines

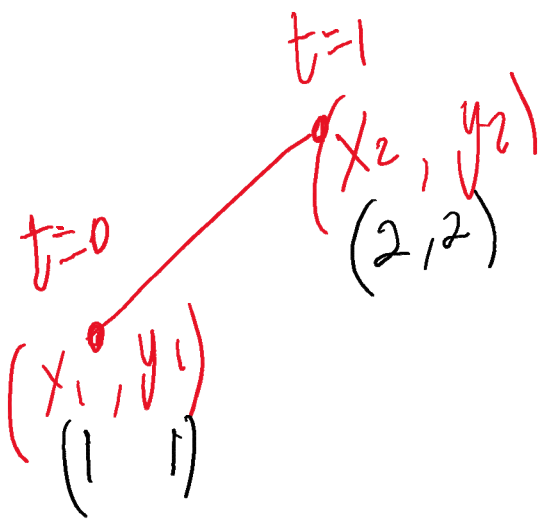
- A line segment with endpoints $P_1: (x_1, y_1)$and $P_2: (x_2, y_2)$ can be defined in the parametric form as follows
- $P = P_1 + t\Delta P$
  - $x = x_1 + t\Delta x$
  - $y = y_1 + t\Delta y,$

  where:
  - $0 \leq t \leq 1$
  - $\Delta P = P_2 - P_1$ (directional vector)
  - $\Delta x = x_2 - x_1,$
  - $\Delta y = y_2 - y_1$

$y = mx + c$

$m = \frac{\Delta y}{\Delta x} \rightarrow = 0 ?$

Parametric form



$t=0: \; x = x_1 + t\Delta x = x_1 + t(x_2 - x_1) = x_1$

$t=1: \; x = x_1 + t\Delta x = \cancel{x_1} + x_2 - \cancel{x_1} = x_2$

$t=\frac{1}{2}: \; x = 1 + \frac{1}{2}(2-1) = 1 + \frac{1}{2} = 1.5$

Computing Intersections

$P_a + t_a D_a = P_b + t_b D_b$

Where P is a point vector and D is a directional vector i.e.

$x_{a1} + t_a(x_{a2} - x_{a1}) = x_{b1} + t_b(x_{b2} - x_{b1})$

$y_{a1} + t_a(y_{a2} - y_{a1}) = y_{b1} + t_b(x_{b2} - x_{b1})$

EXAMPLE

$P_{a1} = (1,2); D_a = (3,4)$

$P_{b1} = (5,1); D_b = (-2,1)$

ANSWER

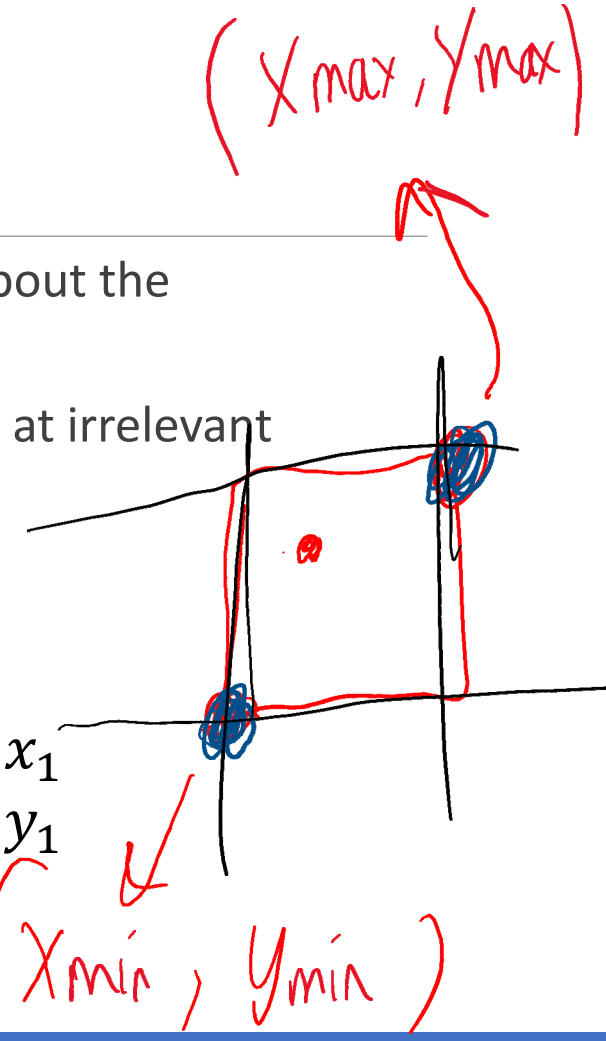
$t_a = \frac{2}{11}; t_b = \frac{-8}{11}$

Intersection = $\left(\frac{17}{11}, \frac{30}{11}\right)$

Liang-Barsky Clipping

- Parametric clipping - view lines in parametric form and reason about the parameter values
- It is more efficient, as it does not compute the coordinate values at irrelevant vertices
- Clipping conditions on parameter:
  - Line is inside clip region for values of t such that:

$$x_{min} \leq x_1 + t\Delta x \leq x_{max} \qquad \Delta x = x_2 - x_1$$
$$y_{min} \leq y_1 + t\Delta y \leq y_{max} \qquad \Delta y = y_2 - y_1$$



Liang-Barsky Clipping

$p_1 = -\Delta x$
$= -(x_2 - x_1)$
$= x_1 - x_2 = 0 - 3$
$= -3$

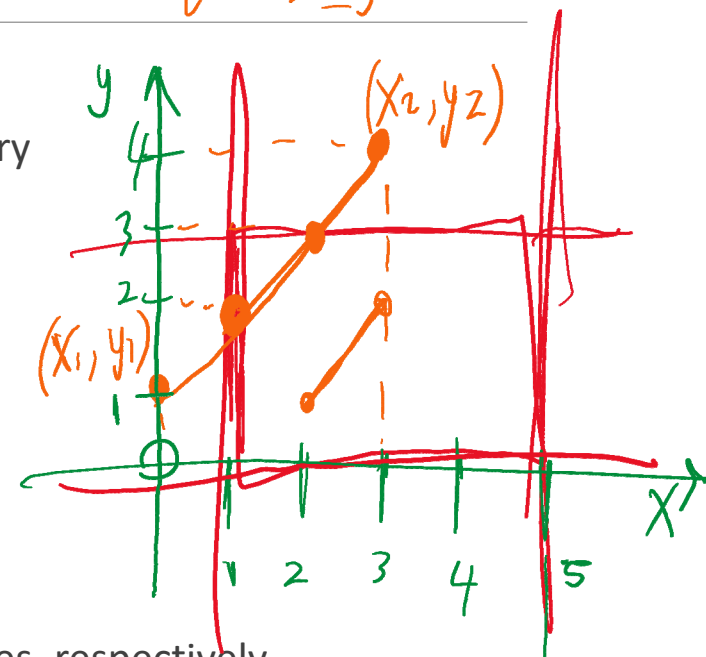
$q_1 = x_1 - x_{min}$
$= 0 - 1$
$= -1$

$t = \frac{-1}{-3} = \frac{1}{3}$

- An infinite line intersects clip region edges when
- t_k represents the parametric value where the line intersects a boundary

$$t_k = \frac{q_k}{p_k} \quad \text{where} \quad \begin{aligned} p_1 &= -\Delta x & q_1 &= x_1 - x_{\min} \\ p_2 &= \Delta x, & q_2 &= x_{\max} - x_1 \\ p_3 &= -\Delta y, & q_3 &= y_1 - y_{\min} \\ p_4 &= \Delta y, & q_4 &= y_{\max} - y_1 \end{aligned}$$

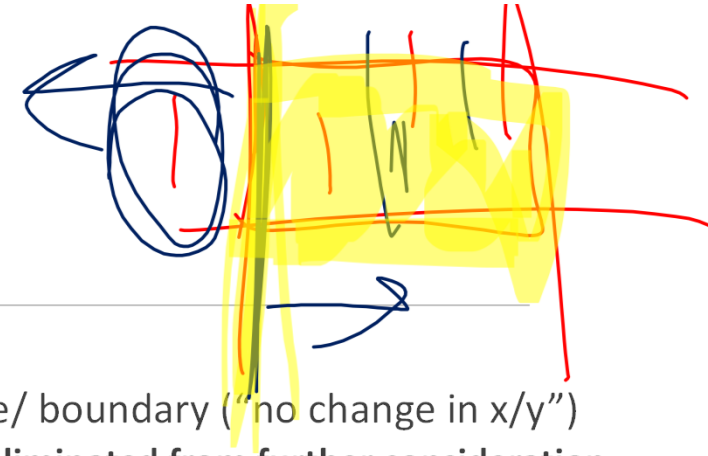
POSITIVE



(x_1, y_1): $(0,1)$; (y_2, y_2): $(3,4)$

- $k = 1, 2, 3, 4$ correspond to the left, right, bottom, and top boundaries, respectively
- t as a time-interval at which the intersection with the clipping window occurs
- Calculate t and then sub to calculate x and y intersection points

Liang-Barsky Clipping…



- If $\boldsymbol{p_k} = \mathbf{0}$, line is parallel to the corresponding clipping-window edge/ boundary ("no change in x/y")
  - If $q_k < 0$, then the **line is completely outside the boundary and can be eliminated from further consideration**
  - If $q_k \geq 0$, (a portion of) the line is inside the parallel clipping border. CONTINUE

$$p_1 = -\Delta x \qquad q_1 = x_1 - x_{\min}$$
$$p_2 = \Delta x, \qquad q_2 = x_{\max} - x_1$$
$$p_3 = -\Delta y, \qquad q_3 = y_1 - y_{\min}$$
$$p_4 = \Delta y, \qquad q_4 = y_{\max} - y_1$$

- When $p_k < 0$, as t increases the line goes from outside to inside - enter
- When $p_k > 0$, line goes from inside to outside - leave
- When $p_k = 0$, line is parallel to an edge (clipping is easy)
- If there is a segment of the line inside the clip region, sequence of infinite line intersections must go: ***enter***, ***enter***, ***leave***, ***leave***

$p_1 < 0 \Rightarrow X_{min} > X_1$

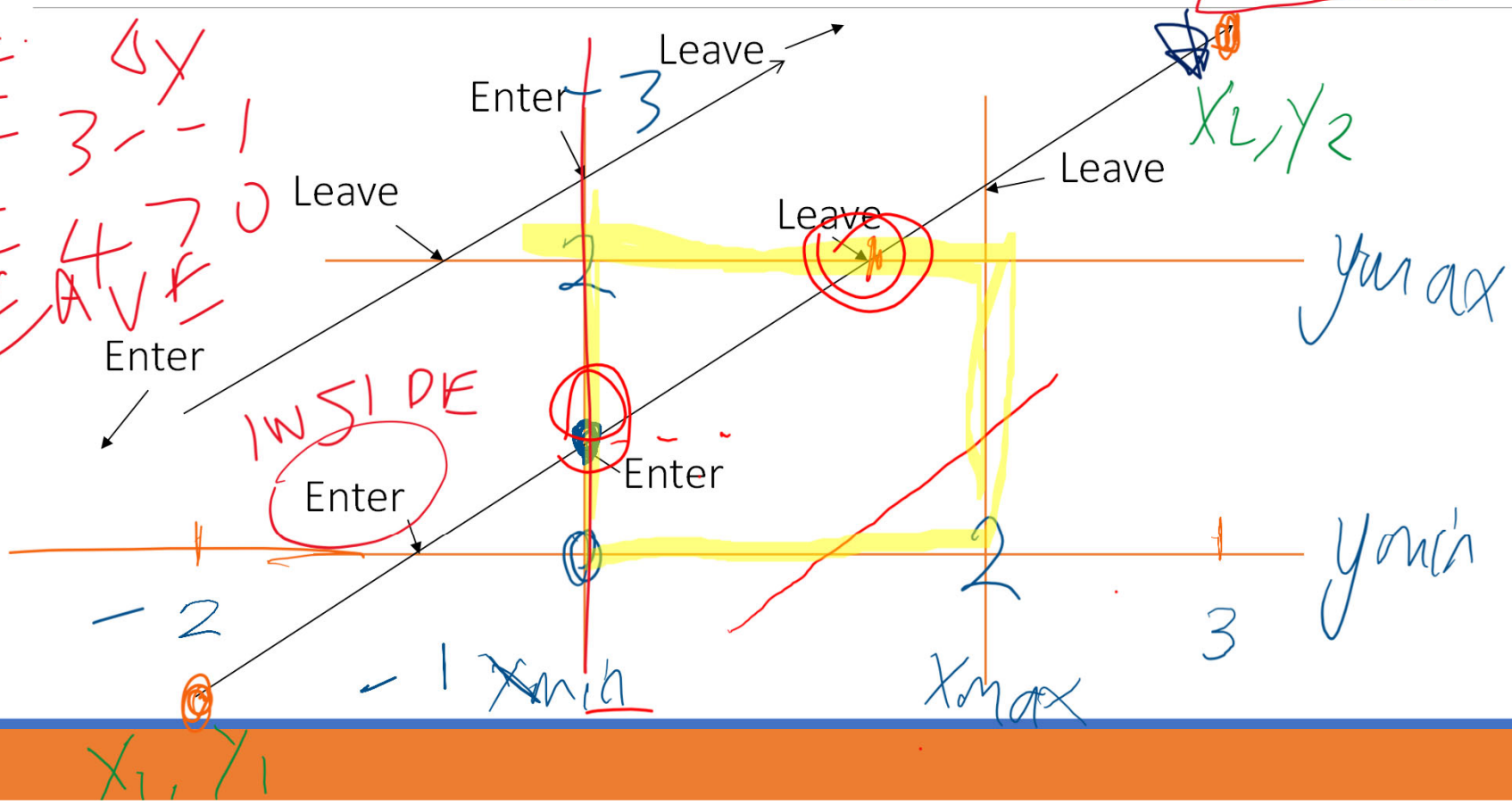
https://www.geeksforgeeks.org/liang-barsky-algorithm/

Liang-Barsky Clipping....

$p_1 = -\Delta x$ (Xmin)
$= x_1 - x_2$
$= -2 - 3 = -5$ ENTER

$\rightarrow$ Ymax

$p_4 = \Delta y$
$= 3 - -1$
$= 4 > 0$
LEAVE



Liang-Barsky - Algorithm

- Compute entering t values, which are q_k/p_k for each $p_k < 0$
- Compute leaving t values, which are q_k/p_k for each $p_k > 0$
- Parameter value for small t end of line is: $t_{small} = \max(0, entering\ t's)$
- parameter value for large t end of line is: $t_{large} = \min(1, leaving\ t's)$
- If $t_{small} > t_{large}$, the line is completely outside the clip window and it can be rejected.
- if $t_{small} < t_{large}$, there is a line segment - compute endpoints by substituting t values
- The visible portion of the line is found by plugging t_{small} and t_{large} into the parametric equation:
  - $P(t_{small}) = P_0 + t_{small}(P_1 - P_0)$
  - $P(t_{large}) = P_0 + t_{large}(P_1 - P_0)$

Liang-Barsky - Example

Consider a line from $P_0(2,3)$ *to* $P_1(8,9)$ against a rectangular clipping window with:

$x_{min} = 1$

$x_{max} = 6$

$y_{min} = 2$

$y_{max} = 7$

- Does the line segment lie within the clipping volume?
- If yes, what are the respective t values of the clipped line?
- If yes, what are the 2 endpoint of the clipped line?

Liang-Barsky Line Clipping: Another View

$x = x_1 + t\Delta x = x_1 + t(x_2 - x_1) = x_1 + tx_2 - tx_1 = (1-t)x_1 + tx_2$

- Suppose we have a line segment defined by end points $p_1 = (x_1, y_1)$ *and* $p_2 = (x_2, y_2)$.

$$\mathbf{p}(\alpha) = (1-\alpha)\mathbf{p}_1 + \alpha\mathbf{p}_2,$$

t

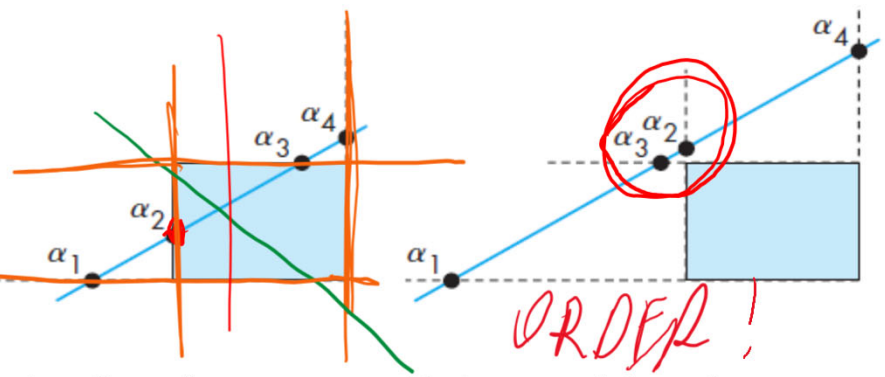
- The line is defined parametrically as follows:

$$x(\alpha) = (1-\alpha)x_1 + \alpha x_2,$$

$$y(\alpha) = (1-\alpha)y_1 + \alpha y_2.$$

- OR

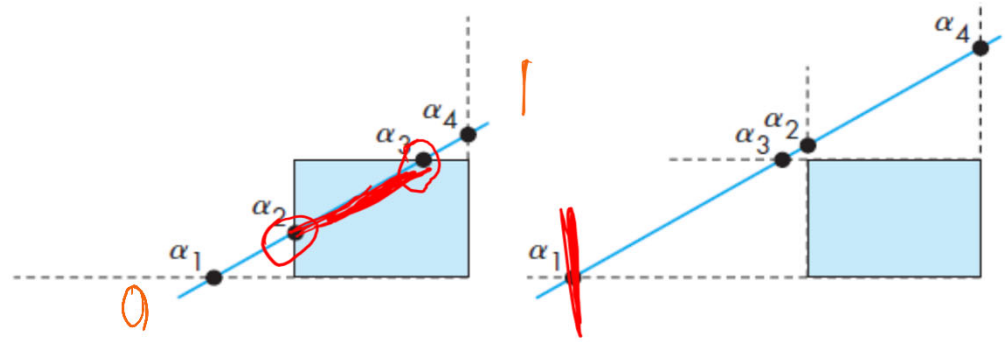
α_1 ymin
α_2 xmin
α_3 ymax
α_4 xmax



ORDER!

- As long as the line is not parallel to a side, there are 4 points that the line intersects with the extended sides of the window

Liang-Barsky Line Clipping: Another View



ORDER

$1 > \alpha_4 > \alpha_3 > \alpha_2 > \alpha_1 > 0$ implies that the line needs to be clipped at points with parameters α_2 and α_3.

$1 > \alpha_4 > \alpha_2 > \alpha_3 > \alpha_1 > 0$ implies the line needs to be discarded without any intersections being computed.

ENTER / LEAVE

Liang-Barsky Clipping: Efficiency

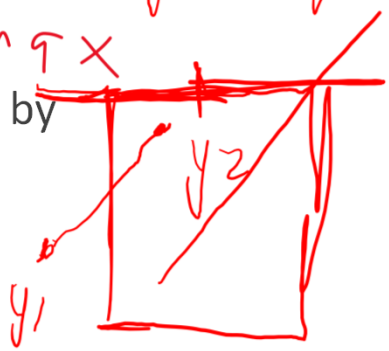
- Efficient implementation of this strategy requires that we avoid computing intersections until they are needed.
- Many lines can be rejected before all four intersections are known.
  - Thus we can avoid floating-point divisions where possible
- At the intersection of the top of the window and a given line, the value of α is given by

$$\alpha = \frac{y_{\max} - y_1}{y_2 - y_1}$$

$y_{max} = y_1 + \alpha \Delta y = y_1 + \alpha(y_2 - y_1)$

$y_{max} > y_2$

X_{max}



- Similar calculation can be done with all 3 sides
- Instead of computing these intersections, at the cost of a division for each, we instead write the equation as

$$\alpha(y_2 - y_1) = \alpha \Delta y = y_{\max} - y_1 = \Delta y_{\max}.$$

- All the tests required by the algorithm can be restated in terms of Δy_{max}, Δy, and similar terms can be computed for the other sides of the windows
- If $\Delta y < y_{max}$, no need for clipping with line $y = y_{max}$

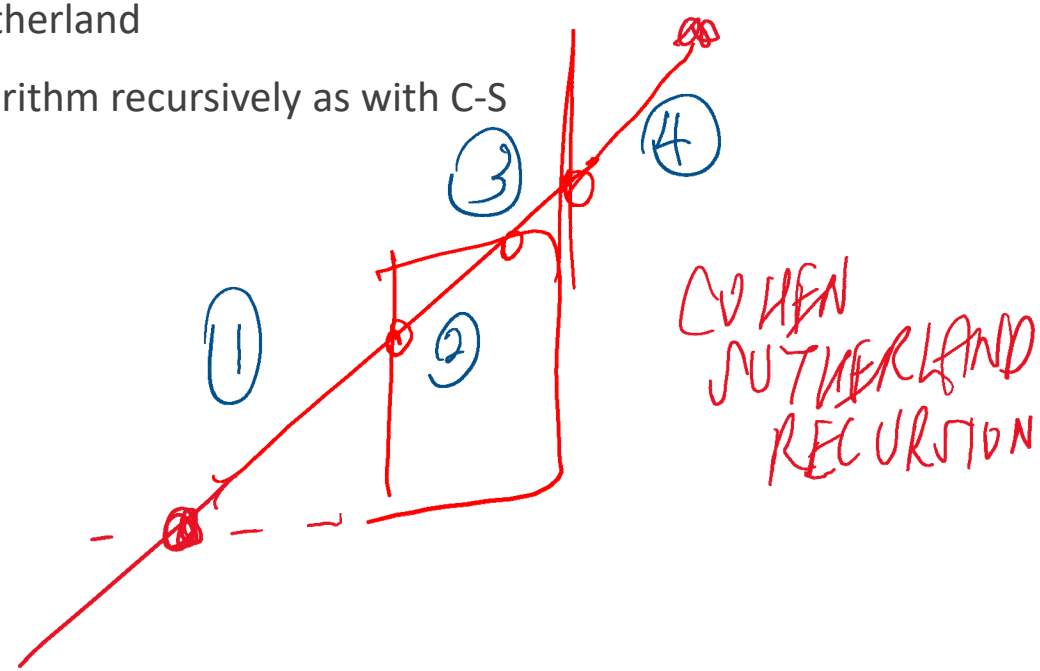
$y_2 - y_1$ BELOW

Liang-Barsky Line Clipping

- In most cases, Liang-Barsky is slightly more efficient
  - As per the Hearn-Baker textbook
  - Avoids multiple shortenings of line segments
- However, Cohen-Sutherland is much easier to understand
  - An important issue if you’re actually implementing the algorithms
- Both algorithms can trivially be extended to 3D

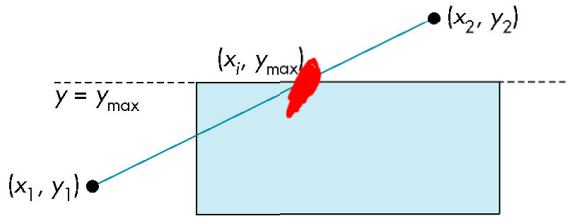
Advantages

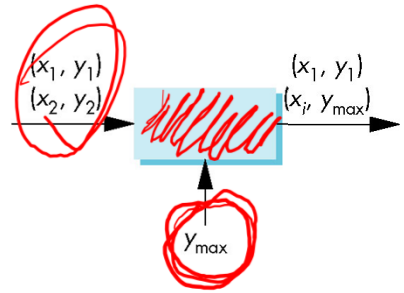
- Can accept/reject as easily as with Cohen-Sutherland
- Using values of α, we do not have to use algorithm recursively as with C-S



Clipping as a Black Box

Can consider line segment clipping as a process that takes in two vertices and produces either no vertices or the vertices of a clipped line segment

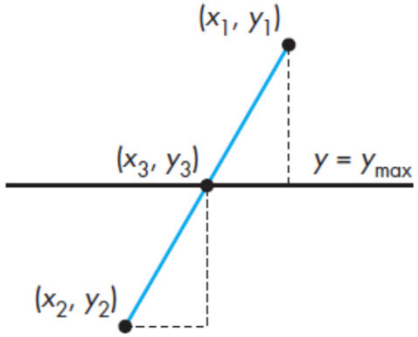




$$y = y_1 + \alpha \Delta y$$

$$x = x_1 + \alpha \Delta y$$

$$\alpha = \frac{y - y_1}{y_2 - y_1} = \frac{x - x_1}{x_2 - x_1}$$

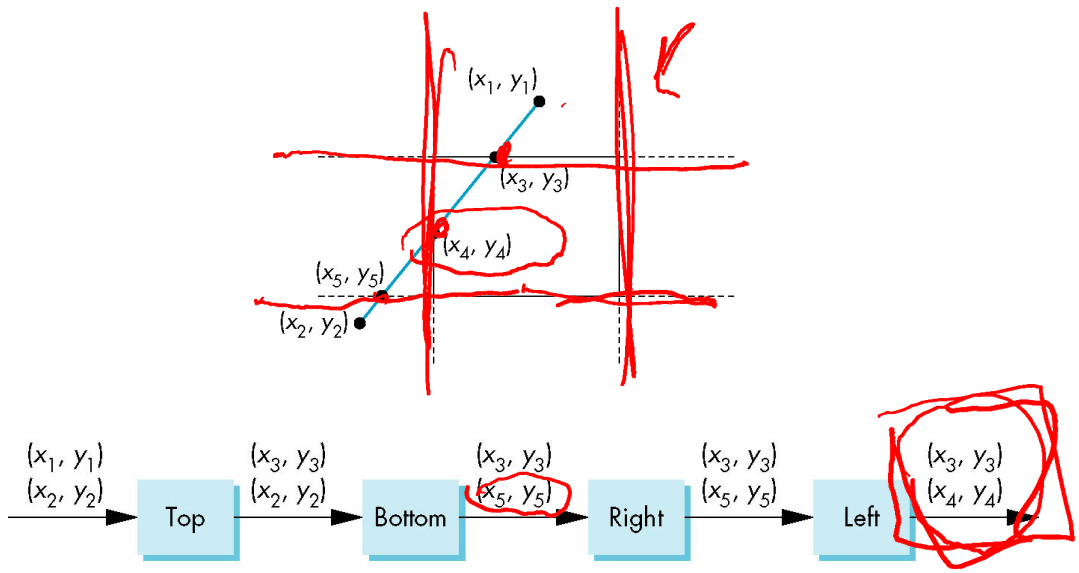


$$x_3 = x_1 + (y_{max} - y_1)\frac{x_2 - x_1}{y_2 - y_1},$$

$$y_3 = y_{max}.$$

Pipeline Clipping of Line Segments

- Clipping against each side of a window is independent of other sides
  - Can use four independent clippers in a pipeline

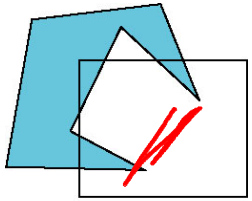


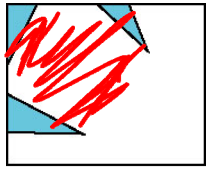
Polygon Clipping

CONVEX

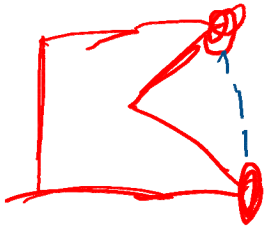
Not as simple as line segment clipping

- Clipping a line segment yields at most one line segment, but…
- …clipping a polygon can yield multiple polygons if the polygon is concave



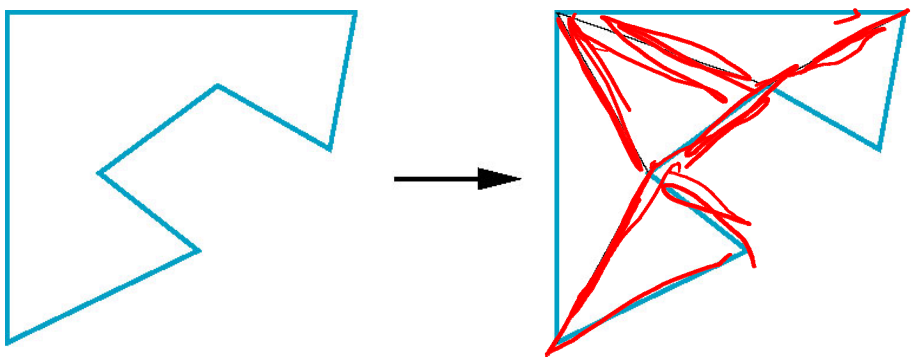


Clipping a convex polygon yields at most one other polygon

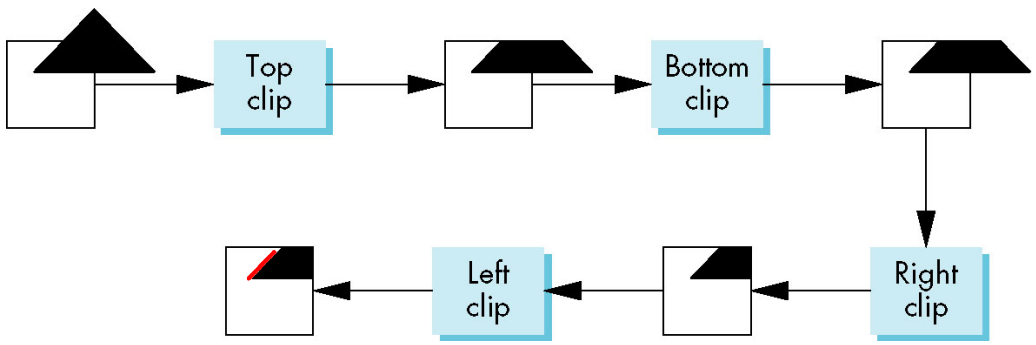


Tessellation and Convexity

- One strategy is to replace nonconvex (*concave*) polygons with a set of triangular polygons (a *tessellation*)
  - Graphics system either forbid the use of concave polygons or
  - divide (tessellate) a given polygon into a set of convex polygons
    - Clipping will now be done on the convex polygons
- In OpenGL tessellation code is in GLU library or with GLSL we use the Tesselation shader



Pipeline Clipping of Polygons

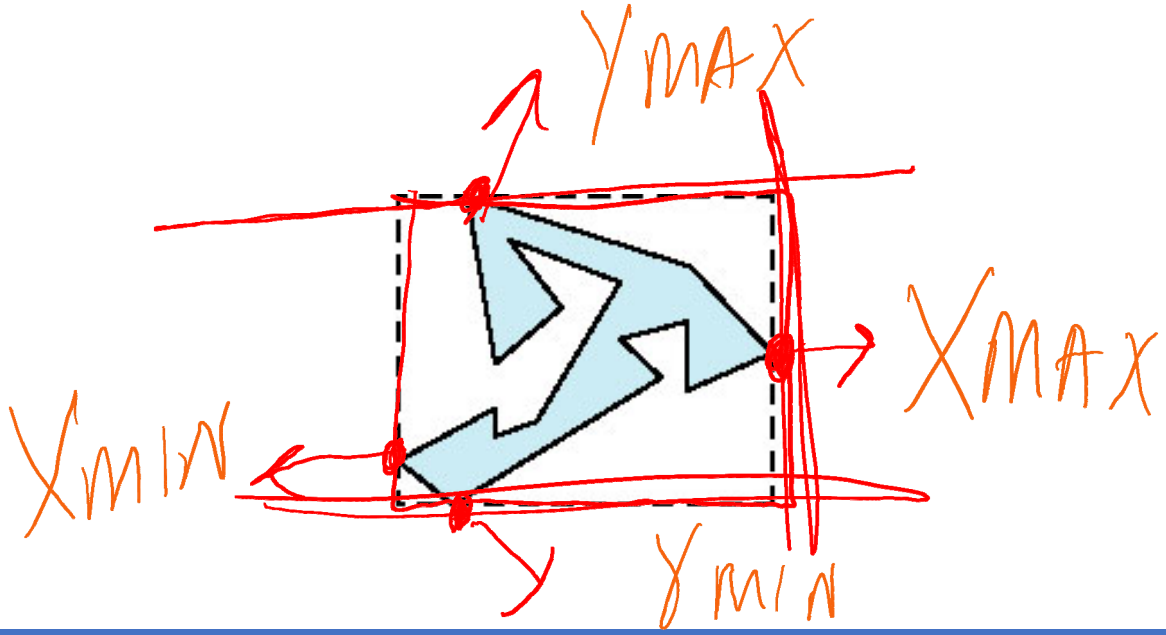


PLANE

- Three dimensions: add front and back clippers
- Small increase in latency

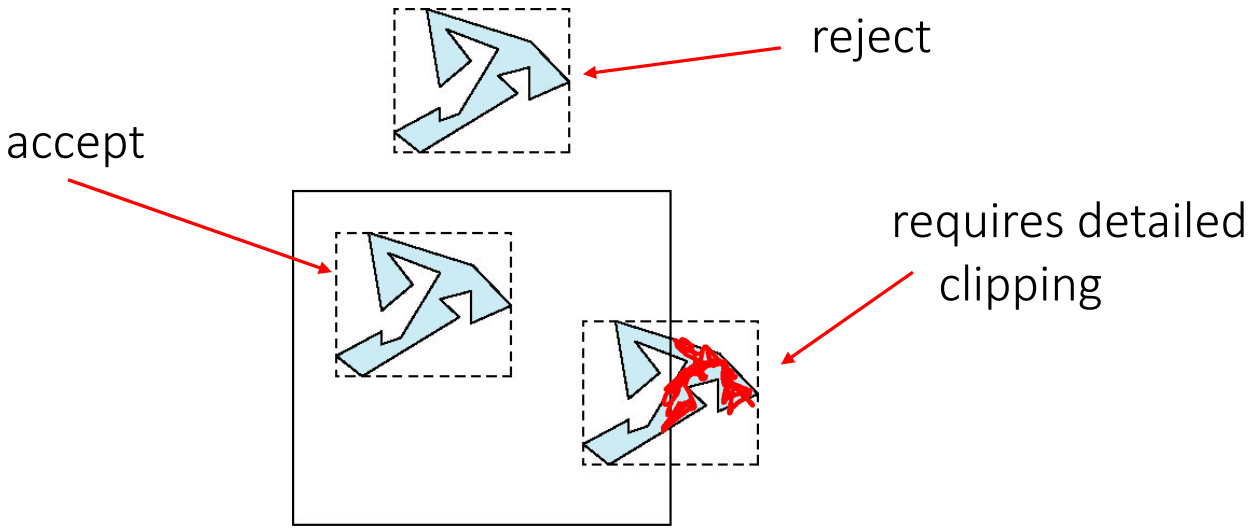
Clipping Other primitives: Bounding Boxes & Volumes

- Rather than doing clipping on a complex polygon, we can use an *axis-aligned bounding box* or *extent*
  - Smallest rectangle aligned with axes that encloses the polygon
  - Simple to compute: max and min of both x and y



Bounding boxes

Can usually determine accept/reject based only on bounding box



Clipping and Visibility

- Clipping has much in common with hidden-surface removal
- In both cases, we are trying to remove objects that are not visible to the camera
- Often we can use visibility or occlusion testing early in the process to eliminate as many polygons as possible before going through the entire pipeline
- However, we will look at rasterization before we go to hidden surface removal.

Additional Clipping Algorithms

- Nicholl–Lee–Nicholl (NLN): Optimisation of Cohen–Sutherland (fewer rejects/tests).
- Cyrus–Beck: Generalises Liang–Barsky (works with any convex polygon)
- Sutherland–Hodgman: Polygon clipping for a convex clipping window.
- Weiler–Atherton / Greiner–Hormann: Advanced polygon clipping (handles concave shapes with holes).
- GPU Hardware Clipping: Done in clip space by the graphics pipeline (modern standard)